



Shri Vile Parle Kelavani Mandal's
INSTITUTE OF TECHNOLOGY
DHULE (M.S.)
DEPARTMENT OF COMPUTER ENGINEERING

Subject : Machine Learning Lab	Remark
Name: _____ Roll No. : _____	Signature
Class: T.Y. Comp Engg. Batch : T Division: _____	
Expt. No. : 05 Date : / /	
Title : Write a program to implement k-Nearest Neighbor algorithm to classify the iris data set. Compute the accuracy of the classifier, considering few test data sets and Visualize model with Python.	

Rubrics :	Date of Submission:		
	Marks Split up to	Maximum Marks	Marks Obtained
	Performance/conduction		
	Report Writing		
	Attendance		
	Viva/Oral		
	Total Marks		
	Aim :	The main aim of this lab is to demonstrate the how to Creating and Visualizing KNN with Python Secondary aim of this lab is to understand - What is KNN? - What KNN is used for - How to Fit a KNN using Scikit-Learn - How to Visualize KNN using Matplotlib	
Software Required :	Jupyter Notebook , Python 3.6, Or Google Colab		
Dataset Used:	Iris Plant Dataset from UCI Machine learning Repository		

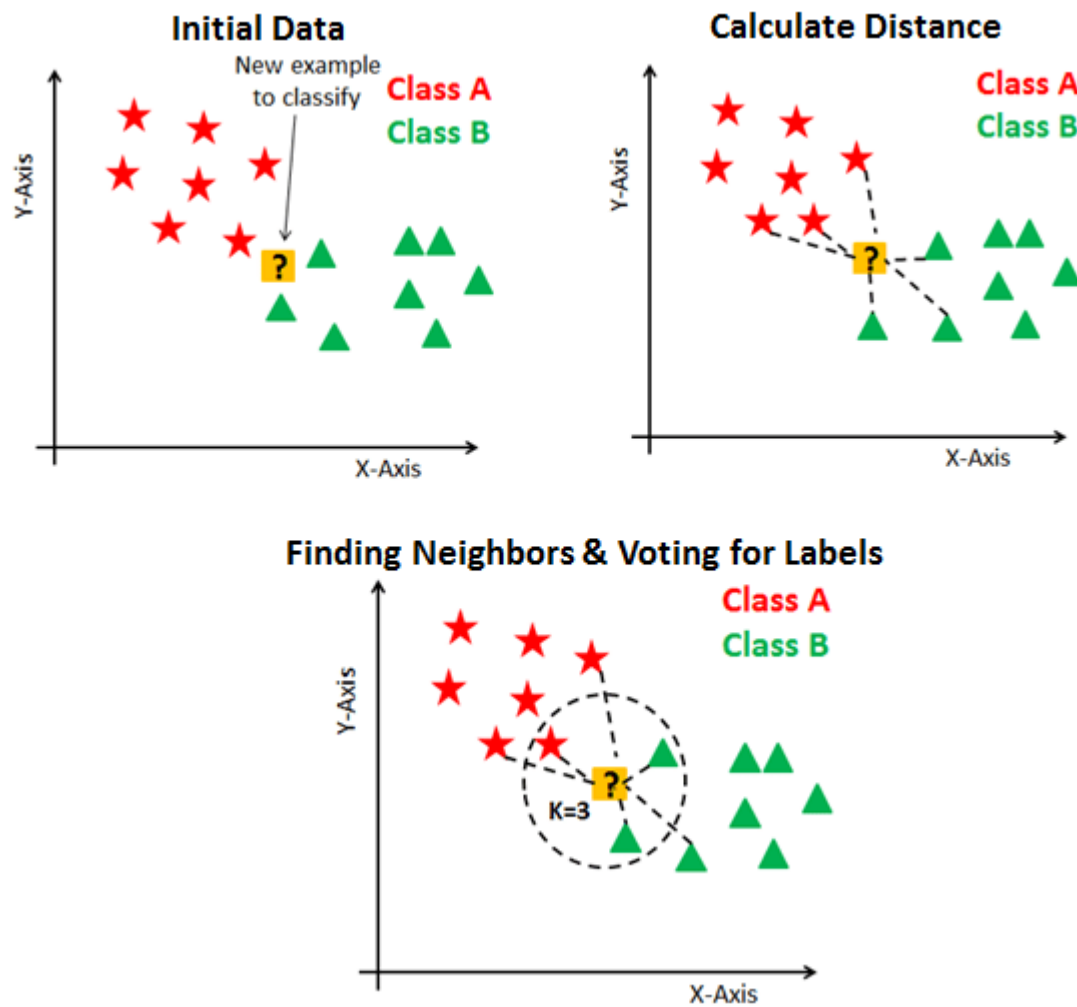
Theory :

Introduction to K Nearest Neighbor

K Nearest Neighbor(KNN) algorithm is a very simple, easy to understand, versatile and one of the topmost machine learning algorithms. In k-NN classification, the output is a class membership. An object is classified by a plurality vote of its neighbours, with the object being assigned to the class most common among its k nearest neighbours (k is a positive integer, typically small). If $k = 1$, then the object is simply assigned to the class of that single nearest neighbour.

In KNN, K is the number of nearest neighbours. The number of neighbours is the core deciding factor. K is generally an odd number if the number of classes is 2. When $K=1$, then the algorithm is known as the nearest neighbour algorithm. This is the simplest case.

Suppose P1 is the point, for which label needs to be predicted.



Procedure:	Algorithm steps STEP 1: Choose the number K of neighbors STEP 2: Take the K nearest neighbors of the new data point, according to your distance metric STEP 3: Among these K neighbors, count the number of data points to each category STEP 4: Assign the new data point to the category where you counted the most neighbors Pros and Cons of KNN Pros 1. It is extremely easy to implement 2. As said earlier, it is lazy learning algorithm and therefore requires no training prior to making real time predictions. This makes the KNN algorithm much faster than other algorithms that require training e.g SVM, linear regression, etc. 3. Since the algorithm requires no training before making predictions, new data can be added seamlessly. 4. There are only two parameters required to implement KNN i.e. the value of K and the distance function (e.g. Euclidean or Manhattan etc.) Cons 1. The KNN algorithm doesn't work well with high dimensional data because with large number of dimensions, it becomes difficult for the algorithm to calculate distance in each dimension. 2. The KNN algorithm has a high prediction cost for large datasets. This is because in large datasets the cost of calculating distance between new point and each existing point becomes higher. 3. Finally, the KNN algorithm doesn't work well with categorical features since it is difficult to find the distance between dimensions with categorical features. Step 1: Importing Libraries Step 2: Importing the Dataset We are going to use the famous iris data set for our KNN example. The dataset consists of four attributes: sepal-width, sepal-length, petal-width and petal-length. These are the attributes of specific types of iris plant. The task is to predict the class to which these plants belong. There are three classes in the dataset: Iris-setosa, Iris-versicolor and Iris-virginica. Step 3: Perform Exploratory Data Analysis Step 4: Preprocessing A. Train Test Split
--------------------------	---

	To avoid over-fitting, we will divide our dataset into training and test splits, which gives us a better idea as to how our algorithm performed during the testing phase. This way our algorithm is tested on un-seen data, as it would be in a production application. B. Feature Scaling Before making any actual predictions, it is always a good practice to scale the features so that all of them can be uniformly evaluated. Step5: Training and Predictions It is extremely straight forward to train the KNN algorithm and make predictions with it, especially when using Scikit-Learn. The first step is to import the KNeighborsClassifier class from the sklearn.neighbors library. In the second line, this class is initialized with one parameter, i.e. n_neighbours. This is basically the value for the K. There is no ideal value for K and it is selected after testing and evaluation, however to start out, 5 seems to be the most commonly used value for KNN algorithm. The final step is to make predictions on our test data. Step 6: Evaluating the Algorithm For evaluating an algorithm, confusion matrix, precision, recall and f1 score are the most commonly used metrics. The confusion_matrix and classification_report methods of the sklearn.metrics can be used to calculate these metrics. The results show that our KNN algorithm was able to classify all the 30 records in the test set with 100% accuracy, which is excellent. Although the algorithm performed very well with this dataset, don't expect the same results with all applications. As noted earlier, KNN doesn't always perform as well with high-dimensionality or categorical features. Step 7: Comparing Error Rate with the K Value In the training and prediction section we said that there is no way to know beforehand which value of K that yields the best results in the first go. We randomly chose 5 as the K value and it just happen to result in 100% accuracy. One way to help you find the best value of K is to plot the graph of K value and the corresponding error rate for the dataset. In this section, we will plot the mean error for the predicted values of test set for all the K values between 1 and 40. To do so, let's first calculate the mean of error for all the predicted values where K ranges from 1 and 40. Step 8: The next step is to plot the error values against K values. Execute the following script to create the plot Step 9: Visualize the KNN
Task to Perform:	1. Implement Decision tree and find the accuracy of your model 2. Visualize your build model in python
Conclusion	In this way we successfully implement and visualize KNN in python.

Test Your Knowledge

1. For the model we created in lab session write the following results

Training Accuracy	
Precision	
Class for Testing Example	{Petal Width = 1.2 Petal Length = 2.0 Sepal Length = 3.2 Sepal Width = 1.2}
F1 Score	

2. What is the best K value for the model you created in during Lab Session?

3. Which Method we used to Find Best K?

4. Create the model with following settings
n_neighbors= Any random Number,
p=2,
metric=use other than 'minkowski'
write a code in your .ipynb file and find following result

Training Accuracy	
Precision	
Class for Testing Example	{Petal Width = 1.2 Petal Length = 2.0 Sepal Length = 3.2 Sepal Width = 1.2}
F1 Score	

5. What is the best K value for the above model?